Reference Summary

Mestrado em Engenharia Informática
Faculdade de Ciências da Universidade de Lisboa

Bruno Faustino
fc59784

2026/2027

Contents

1 Rust Exceptions - Robust and Size-Optimized Error Handling in Rust for Resource-Constrained Embedded Systems **2**

2 Rust Channels **3**

3 Java and .NET Exceptions - Exception Handling: A Field Study in Java and .NET **4**

4 Java Exceptions and Channels **5**

5 Java Exceptions - Program development in Java **6**

6 Kotlin Exceptions and Channels **8**

7 Scala Exceptions **10**

8 Scala Channels **11**

9 Haskell Exceptions and Channels **11**

10 OCaml Exceptions **13**

11 Go Exceptions and Channels **14**

12 Python Exceptions **15**

13 Erlang Exceptions - Programming Erlang - Software for a Concurrent World **16**

14 Elixir - Programming Elixir 1.6 - Functional | Concurrent | Pragmatic | Fun **18**

15 FreeST Syntax - Polymorphic lambda calculus with context-free session types **19**

16 Conclusions **19**

1 Rust Exceptions - Robust and Size-Optimized Error Handling in Rust for Resource-Constrained Embedded Systems

This paper discusses the differences between panic-based and result-based exception handling in Rust, focusing more on embedded systems, where software-based fault handling is essential due to lack of hardware redundancy and OS support.

Rust's compiler profiles (dev, release, test, bench, custom) offer a way to change compiler settings like optimizations and debugging symbols. Compiler profiles have a significant influence on performance and code size. The paper evaluates the binary size and memory use of different exception handling strategies and compiler profiles.

3 examples are used for the experiment, representing a balanced set of workloads common in embedded systems: computational, peripheral and memory-centric. For each example, 3 exception handling strategies are applied: no handling (baseline), return-based (Result<T, E>) and panic-based (panic-halt handler). For each, 6 different compiler profiles are used, totaling 54 binaries.

The results show that the binary size between no exception handling and result-based is almost identical, even matching in compiler profiles with link-time optimization. This suggests that the addition of the boilerplate code for exception handling does not directly correlate with the size of the compiled binary. Meanwhile, panic-based consistently generated the largest binaries in every example and profile combination due to the inclusion of Rust's internal panic-handling routines, panic formatting mechanisms and additional braching from panic paths.

While careful design of error handling contributes to efficiency, the broader optimization context defined by the build profile remains the determining factor.

The result-based model offers a practical balance between safety, determinism and minimal footprint, making it the most suitable strategy.

Panic Handlers

Reference (ACM): Philipp Schloyer, Bernhard Bauer, and Dominik Merli. 2026. Robust and Size-Optimized Error Handling in Rust for Resource-Constrained Embedded Systems. In 19th European Workshop on Systems Security (EuroSec'26), April 27–30, 2026, Edinburgh, Scotland Uk. ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3803525.3804990

2 Rust Channels

- Asynchronous channels by default.
- Limited capacity: sender blocks if buffer is full.
- On the receiver side, calls to close or the receiver handle dropping return an error in the sender side.
- On the sender side, Result<(), SendError<T> > is returned, which can be matched in a case statement, allowing for the error cases to be handled.
- Matching on the Result and then matching on the specific error is a lot of boilerplate. Function unwrap_or_else removes the need for the first match. Other variants such as unwrap and expect are also helpful: they return the Ok branch or call panic!, while expect also receives the panic message as argument, but these do not allow error handling.
- The ? operator can be used to immediately propagate the error outside the function or continue if successful (methods can be chained with the ? operator).
- Function try_send does not block if the buffer is full, returning an error.
- Function send_timeout is like try_send but first waits a given time before returning an error if the buffer is still full or the receiver is dropped.
- Macro select evaluates multiple async branches and returns the first that completes.

References:

- Error Handling

- STD Channels
- Tokio Channels

3 Java and .NET Exceptions - Exception Handling: A Field Study in Java and .NET

- In .NET the programmer is not forced to declare which exceptions can occur or deal with them, unlike Java: it does not have checked exceptions.
- Being forced to think about errors and cathing them in a generic block can get the system in a corrupt state which is harder to debug, instead of simply terminating the application.
- Main problems designing exception structures: global flow of exceptions and unanticipated exceptions.
- Applying error handling code as an advice (in AOP terminology) tried to provide a clear separation between business code and exception handling code, and reduce the amount of code for exceptions. Handler reuse is more difficult than usually advertised - neither of the following aspects improved much: separation of concerns, coupling between components and depth of inheritance tree, cohesion in the access to fields by pairs of method and advice, and dimension of code.
- Total amount of code used for handling exceptions is lower than expected because it is normally just used to alert the user, abort applications or force them to continue their executions, instead of actually recovering from the errors, even in mature applications.
- Most common error handling: empty, log, change object state configuration, throw, continue loop (try again), return (or break), rollback try block, close, assert (new exception thrown if assert fails) and delegate (only for .NET).
- Empty error handling isn't only used by lazy programmers so their programs compile. It is also used to avoid premature termination from non-fatal exceptions.
- Log and throw are often used together.
- finally handlers are often used to close connections and release resources, along with file deletion, event firing, stream flushing and thread termination.

- For the most part, empty and log are the most common in catch handlers, and close is the most common in finally handlers.
- The most common exception classes used in the catch statements are the generic ones, which can be explained by some of the most common exception handling being log and return.
- Most Java exceptions are caught after 1 level in the stack, while .NET exceptions are more sparse, possibly being caught after 17 levels, since programmers are not forced to catch checked exceptions.
- Most protected blocks have only 1 catch handler.
- Programmers are concerned with throwing the exception objects that best fit the particular situation, but are not so concerned with implementing handling code.
- The runtime should try to deal with the problems automatically whenever possible to free the programmer from doing so (without care...): https://www.researchgate.net/publication/231240159_Making_Exception_Handling_Work.

Reference: Bruno Cabral and Paulo Marques. 2007. Exception handling: a field study in Java and .NET. In Proceedings of the 21st European conference on Object-Oriented Programming (ECOOP'07). Springer-Verlag, Berlin, Heidelberg, 151-175. https://dl.acm.org/doi/10.5555/2394758.2394770

4 Java Exceptions and Channels

- JVM waits for all non-daemon threads to end, even if the main thread has already ended.
- Unhandled exceptions are printed to the terminal by default, along with the stack trace.
- Exceptions in a child thread do not propagate to the main thread, but terminate the child thread and print the exception and stack trace.
- Channels can read into (write) or out of buffers. The operation is always read. The buffer needs to be flipped to switch between read and write.
- Data remains in the buffer after reading: it needs to be freed with clear or compress.
- Some data can never be read.

- There are many types of channels (for files, sockets, ...).
- Channels can usually be in blocking or non-blocking mode.
- Non-blocking mode uses Java's Future class. Alternatively, a CompletionHandler can be used during write and read operations, which needs 2 methods to be implemented: one if the operation succeeded (completed) and another if it failed (failed). The last one receives the exception that was thrown as argument.

References:

- Exceptions on the main method
- Exceptions on a child thread
- Java Channels

5 Java Exceptions - Program development in Java

- Partial procedures are allowed to only use a subset of the arguments and ignore the others, while the caller has to make sure it does not provide arguments outside of this subset.
- There is no guarantee that the arguments are in the subset so the procedure may loop forever or return an error result (which can be hard to debug).
- The program should continue after the error in a similar way or at least halt with an informative error message, without causing damage to permanent data.
- Total procedures should be used, informing the caller if the arguments are not in the subset necessary for the procedure to perform its function.
- Returning a particular result is not a good option since the caller could ignore it by mistake or there might not even be a possible particular result available (when any result is possible).
- An exception mechanism solves these problems and also allows the handling of these situations to be separated from the normal program control flow.
- Procedures are documented with Requires, Modifies and Effects sections.

- Checked exceptions need to be declared along with the procedure and code that calls the procedure needs to handle it. Failing this causes a compile-time error.
- Unchecked exceptions are a subclass of RuntimeException and do not need to be declared with the procedure or handled (but they can).
- When a procedure ends with an exception, control is given to code that handles the exception.
- An exception is handled with the try-catch statement.
- Multiple catch statements can be provided for the same try and multiple try statements can be nested. Exceptions not handled in an inner try can be handled by an outer try.
- A procedure that calls another procedure that throws an unhandled exception propagates the exception if either the caller declares the exception or a supertype of it in its header, or the exception is unchecked. Otherwise there is a compile-time error.
- To know the origin of an unchecked exception, we should narrow the scope of the try statement, so it has less code and the exception location is easier to debug.
- Exceptions can be handled specifically or with a generic action, like reporting to the user and shutting down or restarting to an earlier state without fixing the exact problem.
- Exceptions can be caught and thrown again with a different type and information.
- Exceptions should be based on the function abstraction and not with how it is implemented.
- Depending on how the exception mechanism is implemented, using them for controlling problem flow could be expensive.
- Exceptions are not the same as errors. They are a mechanism that allows a method to bring some information to the attention of the caller.
- The meaning of exceptional is arbitrary.
- Exceptions should be used to eliminate most Requires clauses. Requires clauses should still be used if a requirement cannot be checked or would be too expensive to check.

- By using an exception, we distinguish normal results from other results. Encoding results may be acceptable in private methods.
- If you expect the caller to avoid calls that raise the exception, then it should be unchecked. Otherwise it should be checked, which means the caller needs to handle it.
- FailureException should be thrown when the code checks an assumption that should hold but it doesn't (defensive programming).
- Checks should always be made unless we can prove the error never happens or the checks are costly.

Reference: Barbara Liskov and John Guttag. 2000. Exceptions. *In Program Development in Java: Abstraction, Specification, and Object-Oriented Design*, Addison-Wesley Professional, Courier Westford, United States, 57–76.

6 Kotlin Exceptions and Channels

- Exceptions are all unchecked by default.
- Similar to Java, exceptions belong to an Exception class that can be extended with new exceptions, which can be thrown and caught. Multiple exceptions can be caught in the same try block (with different catch blocks). finally block is executed after either the try or catch blocks finish.
- Overall result comes from either the try or catch blocks. The finally block does not change the result.
- Classes that implement AutoClosable provide the use function that closes the resource after the specified block of code is executed.
- Built-in functions require, check and error reduce some of the usual boilerplate code. require verifies the arguments passed to the function throwing an exception if it fails. check verifies the state of an object, throwing an exception if it fails. error throws an exception if reached. The first two allow the compiler to perform smart casting, casting the variable used to a non-nullable type if the checks are successful.
- The result of the try-catch block can be assigned to a variable.
- The result of throw has type Nothing, which is a subtype of all other types. If the compiler infers a Nothing type in the function signature, it warns the user (unless the function explicitly returns Nothing).

- Provides compile-time warnings for potential runtime errors.
- Uncaught exceptions print a message in the terminal and terminate the thread.
- Kotlin uses coroutines which are similar to threads but they dont need to execute on a particular thread, making them more lightweight.
- Main coroutine is launched from GlobalScope.
- Uncaught exceptions in child coroutines are propagated to the parent until they reach the main coroutine.
- If the main coroutine does not catch an exception, the UncaughtExceptionHandler kicks in like Java, printing a message and crashing the app.
- CoroutineExceptionHandler allows the programmer to specify an exception handler that is executed when all children of the main coroutine finish executing and any threw an exception, instead of calling the UncaughtExceptionHandler. Here the first exception is caught and all other exceptions come with it as supressed. If the child coroutine was launched with async and throws an exception, even if the exception is caught in the parent after calling await, it still goes up until the CoroutineExceptionHandler (but the parent continues execution).
- When child coroutines fail due to an exception, they throw CancelledException, which can be caught by the parent during a join with the child. The parent has access to the specific exception thrown in the child.
- Child coroutines fail silently if they are async and no await is called.
- Program terminates when the main coroutine ends, even if there are other coroutines running.
- Channels are used to communicate between coroutines.
- Channels are synchronous if no buffer size is specified and asyncronous otherwise.
- On channel close, a token is sent to the buffer.
- Channels can be accessed by multiple producers and consumers.
- Exceptions are thrown if send is invoked on a closed channel or receive is invoked on a closed channel with an empty buffer.

References:

- Kotlin Exception Handling
- Kotlin Coroutines
- Exception handling in Kotlin coroutines 1
- Exception handling in Kotlin coroutines 2
- Kotlin Channels
- Kotlin Channels Documentation
- Kotlin Main Thread

7 Scala Exceptions

- Has try-catch-finally like Java and Kotlin, but the syntax is more similar to the match syntax.
- Has Option class with Some and None subclasses, which can be used in pattern matching.
- Either class is also available, and Left can have a subclass of exception.
- Also has for...yields... blocks. If the code in the for block succeeds, the yield block is executed and its result is wrapped in Some. Otherwise, the result is None.
- Another alternative is using datatype Try with instances Success and Failure, which is easier for functional composability than try-catch since the exception can be handled later.
- Catch objects can be used to centralize exception handling code and avoid repeating code. They can be used with Try to match specific exceptions and handle them in already defined ways, or ignore specific exceptions.
- Exceptions are printed to the terminal when uncaught.
- An exception in a child thread does not stop the parent thread.
- Running a child thread should be wrapped in a future, so that the parent can catch exceptions in the child thread if needed.

References:

- Functional Scala Exception Handling
- Scala Exception Handling

8 Scala Channels

- Scala does not have built-in channels, but the Ox library provides channels to Scala that are similar to Go.
- Sending and receiving are blocking operations. If the buffer has a capacity larger than 1, sending can be non-blocking until the buffer is full.
- done is used to close a channel and error is used to signal an exception.
- receive returns an Either with Left having a ChannelState, which can be Done or Error. Exceptions are handled in a match expression.

Reference: Scala Channels

9 Haskell Exceptions and Channels

- Exception is a type class that can be instantiated with new custom exceptions. In a datatype used to instantiate Exception, all of its constructors are used as new exceptions.
- SomeException is the generic exception and SomeAsyncException is the generic async exception.
- Exceptions can be thrown from IO operations, pure code (function evaluate must be called on the pure code) and external events (asynchronous).
- Library Control.Exception offers functions try, catch and finally. Function handle is similar to catch but with the arguments swapped.
- Uses the IO monad.
- Only main difference between try and catch is that the handler used in catch cannot be interrupted by an asynchronous exception. Raising an asynchronous exception (with throwTo) blocks until the catch handler completes.

- Function throwTo is used to throw an exception in another thread (asynchronous).
- Functions bracket, finally and onException are similar and allow a second computation to be performed, even if the first raised an exception.
- Threads are independent but there is a main thread.
- The program terminates when the main thread finishes executing. All other threads will terminate as well.
- Default uncaught exception handler logs an error message based on the Show instance. It can be changed in all threads with function setUncaughtExceptionHandler.
- The application crashes if an uncaught exception is thrown in the main thread.
- Exceptions in the uncaught exception handler terminate the main thread with an error message, or get stuck in a loop if it happened in another thread, which eventually terminates.
- Exceptions are not passed to the parent thread.
- Alternatively, exceptions can go in the Left of the Either monad and a case statement can be used to handle exceptions.
- Exceptions can only be caught in IO code.
- Channels also use the IO monad and are unbounded.

References:

- Haskell Impure try catch handle
- Haskell Exception Handling
- Haskell Exception Handling with Either
- Haskell Threads
- Haskell Uncaught Exception Handler
- Haskell Threads Docs
- Haskell Channels

Related Papers:

- Asyncronous exceptions in Haskell
- A semantic for imprecise exceptions

10 OCaml Exceptions

- Exceptions do not appear in types, but they are compiled into efficient machine code.
- Exceptions belong to an extensible sum type exn and are instantiated with the keyword exception in a way similar to defining a datatype.
- Exceptions are caught with try-with, similar to try-catch, where the with block is similar to a case block with exceptions.
- Has functions invalid_arg and failwith that automatically raise the predefined exceptions Invalid_argument and Failure due to invalid parameters and results that could not be produced, that receive a string for the log message.
- Module Fun has function protect which receives the finally and try-catch blocks, which works like other languages. If an exception is thrown in the finally block, it is wrapped inside the exception Finally_raised to distinguish it from an exception in the other blocks.
- Asynchronous exceptions are thrown when an external factor does not allow the execution of the program and it usually does not make sense to handle them in the program. Examples are Out_of_memory, Stack_overflow and Sys.Break.
- Module Printexc allows access to the message and stack trace of an exception and defining an exception's message.
- Type option works like Haskell's Maybe and has functions map and bind that reduce nested match cases. The downside compared to exceptions is that it does not inform about the reason for returning None.
- Type result works like Haskell's Either and Rust's Result and has functions map, map_error and bind, similar to option. The left side of result can be of type exn, making it also a good strategy for dealing with exceptional situations.
- Default uncaught exception handler that prints the exception (can be changed).
- Exceptions in a child thread are not propagated to the parent thread.

- Program terminates when the main thread finishes executing and all other threads are interrupted.

References:

- OCaml Exception Handling
- OCaml Threads
- Ocaml Main Thread

11 Go Exceptions and Channels

- Errors are values returned explicitly from functions, instead of being hidden in exceptions. Any type that implements method Error of interface error can be considered an error.
- Function fmt.Errorf allows errors to be wrapped inside other errors (using %w).
- Functions return a pair with the result and an error (which can be nil, meaning no error).
- Structs can be defined as errors and can include an enum with the cause of the error.
- Handling of errors is done with ifs.
- Function errors.Is traverses the error tree and returns true if any error matches the target error. This is useful because we lose error equality when wrapping errors inside other errors. Aditionally, the Is function can be overwritten to change what it means for the errors to be equal.
- Function errors.As is similar, but it works with types instead of values. If it finds an error in the error tree with the given type, it assigns that error value to the provided pointer. It can also be overwritten using reflection.
- Function errors.Join concatenates multiple errors, separating them by new lines. The resulting error still Is each one of the joined errors.
- Program terminates when the main thread terminates.
- Error handling while using goroutines and channels needs a separate channel dedicated to receiving possible errors of the program. This

introduces a lot of boilerplate. An alternative is using golang.org/x/sync/errgroup, which allow for the creation of an error group and launching goroutines from a function that can return an error, associating the function/goroutine with the error group. The error group can be waited for, returning all the errors that ocurred in the coroutines of the group, if any.

References:

- Go Exceptions
- Go Functions errors.Is and errors.As
- Go Function errors.Join (with Is and As again)
- Go Error Handling in Channels
- Go Main Thread

12 Python Exceptions

- Uses exceptions like Java.
- Exceptions go up the call stack until a suitable handler for it is found. If none is found, the program terminates and an error message is printed.
- Exceptions are thrown with raise.
- Exceptions are caught and handled with try-except, similar to Java's try-catch. An except without an explicit exception catches any exception. Multiple exceptions can be caught in the same except block.
- The complete exception handling syntax is try-except-else-finally. try, except and finally work like try, catch and finally in Java. The else block executes only if no exception was raised in the try block, while finally always executes.
- Exceptions can be re-raised inside an except block for an outer exception handler to catch. Here, raise does not need to receive the exception again.
- try-except can be used in asynchronous context in the same way.
- Custom exceptions can be made by creating a new class that inherits from one of the built-in exception classes or the base Exception class.

- Honeybadger, Sentry and Datadog are error monitoring services which allow real-time monitoring and visualization of the exceptions that are raised by the application.
- Non-daemon child threads keep executing after the main thread completes.
- Child threads do not propagate exceptions to the main thread. It can be possible through some sort of message passing.

References:

- Python Exception Handling
- Python Threads

13 Erlang Exceptions - Programming Erlang - Software for a Concurrent World

- Error handling is essential for robust distributed applications.
- "Crashing" means the system raised an exception.
- Exceptions are raised implicity due to an internal error or explicitly with built-in functions (BIFs) throw, exit or erlang:error.
- Erlang uses try...of-catch-after-end to handle an exception, or enclosing the expression in a catch expression. It works like a case expression with more blocks. The after block is like the finally block in Java and its result is lost. The of block tries to match the result of the try block with the given patterns (if no exception was raised).
- The of and after blocks can be omitted.
- The BIFs are used in the patterns of the catch block to distinguish between the different exception forms.
- The primitive catch expression transforms an exception into a tuple that describes the error, but loses precision in analyzing the caause of the problem when compared with try-catch. It is better to use try-catch, which was introcuded later to the language.
- BIF exit terminates the process and broadcasts a message to all linked processes.
- BIF throw is used when the caller may want to handle the exception.

- BIF erlang:error is used for crashing errors that callers are not expected to handle and it includes the stack trace in its error message.
- These BIFs can receive an atom that represents the exception type to thrown, which improves error messages and can be matched appropriately by code on the caller side.
- try-catch has a value since everything in Erlang are expressions, so it can be assigned to a variable.
- Alternatively, when error returns are common, a pair can be returned from the function which is either {ok, Val} or {error, Why} and the caller code either uses a case-of expression to handle the error like Rust's Result, or ignores the error case, which raises an exception if the function returns {error, Why}.
- The latest stack trace can be obtained by calling erlang:get_stacktrace while catching an exception.
- Erlang has no shared memory. Processes have a private state and they communicate by sending messages. For a process to check if the other process received a message, it needs to send it a message asking if it received and understood it.
- Pairs of processes can be linked together and if a process dies, it will send the reason of its death to the linked process.
- Primitive spawn spawns a new process and returns its Pid, used to send messages to the process.
- Function self returns the Pid of the current process, used so that the receiver can send a message back to the sender.
- Processes can be registered with a name (atom) and its pid so that any other process can communicate with it.
- Primitive receive waits for any message that matches one of the specified patterns. If the sender crashes or is for another reason unable to send a message, receive can have an after block as well, which includes a timeout in milliseconds to wait for a message, and if none arrives in that time, it executes the after block.
- BIF link links 2 processes. If one dies, it sends a message to the other explaining why. To be more precise: a dying process broadcasts the exit signal to all processes in its link set. link raises an exit noproc exception if the Pid of the process to link to does not exist. BIF spawn_link automatically links the newly spawned process to the current process.

- BIF process_flag(trap_exit, true) turns the process into a system process, which does not terminate when a linked process exits, and it can process the exit reason of the dead linked process.
- Remote handling of errors -> errors in one process can be handled in a different process that can even be in a different machine.
- A process that successfully evaluated its function will exit with reason normal. Linked processes that are not system processes don't die if the reason is normal.
- A process can explicitly send an exit signal to another process like it is dying (has some similarities with Haskell's throwTo function).
- A process can send a kill exit signal, which kills the process it is sent to, even if they are system processes (only exit/2 does this; exit/1 can be trapped by linked system processes). The dying process broadcasts killed instead of kill to avoid the death of more processes than intended.
- A monitor is an asymmetric link.
- Server sockets are automatically closed if the server exits, and clients receive a message {tcp_closed, Socket}.

Reference: Joe Armstrong. 2007. *Programming Erlang - Software for a Concurrent World*. Pragmatic Bookshelf, United States of America.

14 Elixir - Programming Elixir 1.6 - Functional | Concurrent | Pragmatic | Fun

- Exceptions in Elixir are for things that should never happen.
- Exceptions are similar to records and are created by defining a module with defexception inside it, which receives a list of fields for the exceptions.
- Has try-rescue-after as usual but it should rarely be used. Instead, errors should be propagated up to a supervising process.
- Functions that can fail (e.g. opening a file) return a pair that can be either {:ok, result} or {:error, message}. A case expression is used to handle the pair like Haskell's Either. If a file is expected to always open successfully, an exception can be raised when it doesn't. Some functions have a variant with an exclamation mark at the end that

either return the result or raise an exception with a meaningful error message (instead of being a match error).

- Exceptions are raised (with raise), but they are rarely caught.
- Block rescue has patterns that can be [exceptions], name in [exceptions] or name (binded to any exception). The handling of an exception can include reraising the exception (with reraise).
- Erlang's throw, exit and erlang:error are also available and are caught in Elixir with try-catch.
- Instead of using and handling exceptions, the function that can fail should instead be isolated in a process and a supervisor process could restart the process if it fails.

Reference: Dave Thomas. 2018. *Programming Elixir* $\geq$ *1.6 - Functional* |> *Concurrent* |> *Pragmatic* |> *Fun*. Pragmatic Bookshelf, United States of America.

15 FreeST Syntax - Polymorphic lambda calculus with context-free session types

- Syntax.

Reference: FreeST Syntax

16 Conclusions

- Exceptions are often a class/datatype that can be extended/instantiated with new exceptions. But FreeST does not have type classes like Haskell.
- Only Java has checked exceptions by the compiler and methods need to declare in their signature all the checked exceptions that the method can throw.
- Rust does not have exceptions and resorts to a Either-like type (Result) for computations that can fail.
- There is a default uncaught exception handler that usually terminates the thread and logs an error message. Each thread can be given its own uncaught exception handler.

- Haskell's throwTo function allows a thread to raise an exception in another thread, using its id. Erlang can also throw an exception to another process in a similar fashion with the exit BIF.

- All languages with exceptions have a try-catch-finally and throw syntax. OCaml uses try-with instead of try-catch and raise instead of throw. Haskell has functions try, catch and finally that receive blocks of code instead of using all 3 like the other languages. Python/Erlang have an aditional optional block - else/after - which is executed after the try block, only if it succeeded.

- Kotlin and OCaml have functions that automatically throw specific exceptions when called, reducing some of the boilerplate code.

- Rust, Haskell, OCaml and Scala have the Maybe/Option type. Maybe/Option can be used to return a Nothing/None result when a computation fails instead of a default value that we could not distinguish from a possible correct result of the function. The downside is that it does retain information about the cause of the error, unlike exceptions. The languages have a way to chain multiple computations that return Maybe/Option, using the wrapped results of the functions if they are Just/Some or immediately return Nothing/None if one of the functions fails, reducing multiple nested match cases.

- Rust, Haskell, OCaml and Scala have the Either/Result type. Exception handling with try-catch provides a better separation of concerns but can be more verbose than the Either/Result variant enhanced with chaining methods.

- Go does not have exceptions. It has errors that are treated just like values and can be wrapped inside other errors. Any type can be used as an error, including structs. Functions that can fail always return a pair with the result and an error, which can be or not nil. If it is not nil, functions errors.Is and errors.As can be used to check if there is an error value (or error type, respectively) inside the error tree. These functions are used in ifs to treat the errors. Errors can also be concatenated with errors.Join. Goroutines can either use a channel reserved for receiving errors or belong to an error group, which allows them to simply return an error if needed since calling Wait on the group will automatically return all the errors in the goroutines.

- Erlang's exceptions are atoms (constants) that can go in a pair {exception, reason}. Erlang allows for a remote handling of errors: processes can be linked together and an error on one process is broadcasted to all processes in its link set. When a process receives this message it is also terminated, unless it was previously set to be a system process, in

which case it can catch the exception during a receive and process it without dying (if the message is not a exit kill signal for this process). Links between pairs of processes can be symmetric or asymmetric.

- In Erlang and Elixir, exceptions are reserved for when the process is not expected to fail (handled, or not, with try-catch), while a pair of either {ok, Val} or {error, Why} is used for functions that are expected to possibly fail (handled with case).
- The proper way of building robust applications in Erlang and Elixir is not to have exception handling but to isolate functions that can fail in different processes and have a supervisor process that restarts its children based on how the system was configured.
- Rust's recv_timeout waits a given time for a message, returning if no message was received in that time with RecvTimeoutError::Timeout. Erlang's receive can have an optional block after which waits a given time for a message and if none arrives, executes the after block.
- In Rust, binary size from result-based handling is identical to no exception handling, while panic-based handling generated larger binaries due to panic-handling routines, panic formatting mechanisms and branching from panic paths.
- Forcing the user to think about exceptions and catching them in a generic block can get the system in a corrupt state instead of terminating the application, making it harder to debug.
- Most common error handling: empty, log, change object state configuration, throw, continue loop, return, rollback, close and assert.
- Most common caught exceptions are the generic ones.
- Programmers are concerned with throwing the exceptions that best fit the situation, but not so concerned with implementing handling code.

1. if children are not linked to main and main does not send an exit kill signal to a child that is not a system process
2. if linked
3. try catch and finally are functions, not syntax of the language
4. unwrap/expect functions, which call panic on Err
5. panic handler: abort, halt, ...

	Java	Kotlin	Scala	Python	Erlang	Elixir	Haskell	OCaml	Rust	Go
Exceptions	✗	✗	✗	✗	✗	✗	✗	✗		
Either / Result / Tuple			✗		✗	✗	✗	✗	✗	
Error Values										✗
try-catch-finally	✗	✗	✗	✗	✗	✗	✗[3]	✗		
match-case			✗		✗	✗	✗	✗	✗	
if-else										✗
Built-in Thrower Functions		✗						✗	✗[4]	
Operation Chaining			✗				✗	✗	✗	
Uncaught Exception Handler	✗	✗	✗	✗	✗	✗	✗	✗	✗[5]	
Children Survive After Main Dies	✗		✗	✗	✗[1]	✗[1]				
Children Raise to Parent		✗	✗		✗[2]	✗[2]				
Raise to Another Thread / Process					✗	✗	✗			